



รายงาน
เรื่อง Crypto Market Streaming Protocol

เสนอ
ผศ.ดร. สุขุมาล กิตติสิน
จัดทำโดย
นายณัฏชนัย เนียมหอม 6710450741

รายงานนี้เป็นส่วนหนึ่งของวิชา 01418351
Computer Communications and Cloud Computing Principles
หลักการสื่อสารคอมพิวเตอร์และการประมวลผลบนคลาวด์
ภาคต้นปีการศึกษา 2569
คณะวิทยาศาสตร์ภาควิชาวิทยาการคอมพิวเตอร์

สารบัญ

1. วัตถุประสงค์ของโปรแกรม	3
1.1 ปัญหาที่ต้องการแก้ · 1.2 โปรแกรมนี้คืออะไร ใช้ทำอะไร · 1.3 ขอบเขต	
2. คุณลักษณะของแอปพลิเคชัน (Application characteristics)	4
3. การเลือก service model ของ Transport Layer	5
3.1 TCP หรือ UDP · 3.2 ทำไมไม่ใช้ HTTP ที่มีอยู่แล้ว	
4. CMSP/1.0 — Application-Layer Protocol ที่ออกแบบ	6
4.1 ชื่อและภาพรวม · 4.2 รูปแบบ message · 4.3 การกำหนดขอบเขตข้อความ	
4.4 Request messages · 4.5 Response messages · 4.6 Push messages	
4.7 Status codes · 4.8 นิยาม field · 4.9 State machine · 4.10 ตัวอย่างการสื่อสาร	
5. การตรวจสอบด้วย Wireshark	10
6. จุดเด่นของ CMSP	12
6.1 ผลการวัดเทียบ HTTP · 6.2 ความสามารถที่ต่างกัน · 6.3 จุดที่ HTTP ยังชนะ	
ภาคผนวก — ตาราง status code ทั้ง 25 ตัว	14

สารบัญรูป

รูปที่ 1 แอปสำหรับเผ่าราคา · รูปที่ 2 โครงสร้าง message · รูปที่ 3 ปัญหาขอบเขตข้อความของ TCP	
รูปที่ 4 State machine · รูปที่ 5 ลำดับการสื่อสารตั้งแต่เชื่อมต่อจนได้ราคา	
รูปที่ 6 หลาย message ใน TCP segment เดียว · รูปที่ 7 3-way handshake · รูปที่ 8 Follow TCP Stream	
รูปที่ 9 ผลการวัดเทียบ HTTP polling	

1. วัตถุประสงค์ของโปรแกรม

1.1 ปัญหาที่ต้องการแก้

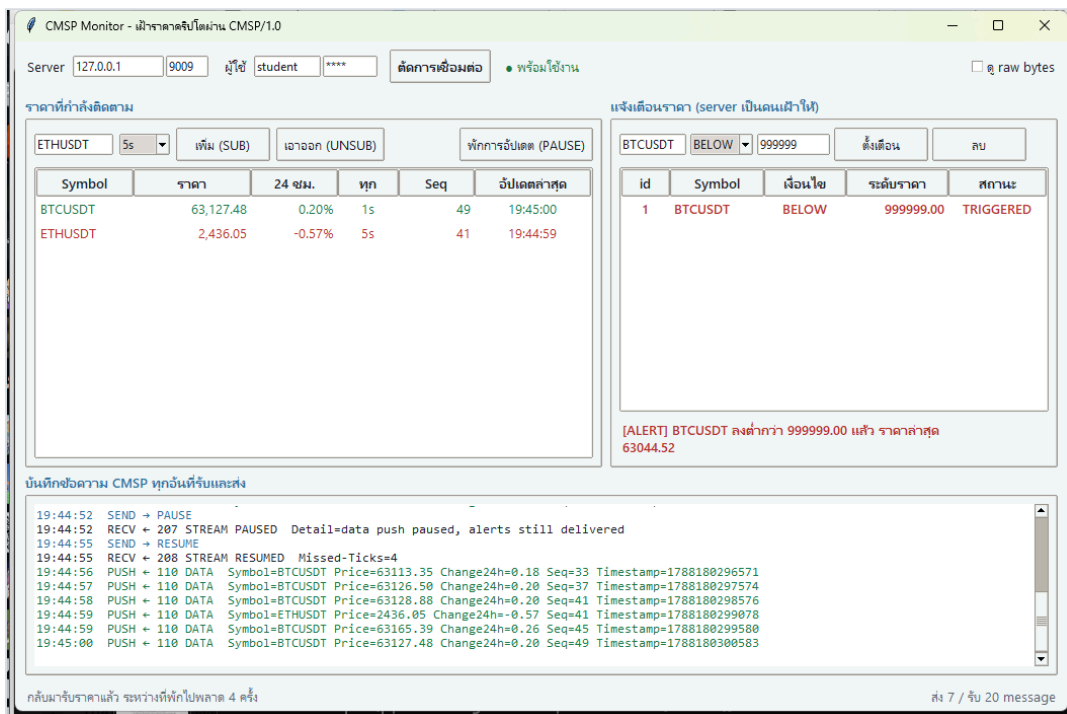
ผู้ที่ติดตามราคาคริปโตเคอร์เรนซีต้องการรู้ราคาทันทีที่มันเปลี่ยน และต้องการให้มีคนเฝ้าให้เมื่อราคาแตะระดับที่สนใจ ถ้าใช้วิธีมาตรฐานคือให้โปรแกรมฝั่งผู้ซื้อค่าขอไปถามราคาเป็นระยะ (polling) จะเจอปัญหาสามข้อพร้อมกัน: ข้อมูลที่ได้มาเก่าไปแล้วเสมอโดยเฉลี่ยครึ่งหนึ่งของรอบที่ถาม, ต้องจ่ายค่าค่าขอซ้ำๆ แม้ราคาจะไม่ขยับเลย, และถ้าอยากตั้งเตือนก็ต้องให้เครื่องของผู้ใช้เปิดค้างคอยเช็คเอง

โครงการนี้แก้ปัญหาด้วยการ **กลไกการส่งสาร**: ผู้ใช้บอกความสนใจเพียงครั้งเดียว แล้ว server เป็นฝ่ายส่งข้อมูลมาให้เองทันทีที่มีของใหม่ พร้อมย้ายงานเฝ้าเงื่อนไขราคาไปไว้ที่ server

1.2 โปรแกรมนี้คืออะไร ใช้สำหรับทำอะไร

เป็นระบบ **streaming server** และ **client** สำหรับติดตามราคาคริปโตแบบทันที ที่สื่อสารกันด้วยโปรโตคอลที่ออกแบบเอง ชื่อ **CMSP/1.0** บน raw TCP socket โดยเรียก socket API เองทุกขั้นตอน ไม่ใช้ framework หรือไลบรารีสื่อสารสำเร็จรูป ระหว่าง client กับ server

ส่วนประกอบ	หน้าที่
CMSP Server	ดึงราคาจาก Binance WebSocket มาเป็นแหล่งข้อมูล แล้ว ไม่ส่งต่อข้อมูลดิบทันที แต่เก็บสถานะของตัวเอง (ตาราง subscription, กฎการแจ้งเตือน, การนับผู้ใช้ต่อ symbol, การควบคุมอัตราการส่ง) ก่อนกระจายให้ client ที่สนใจ
CMSP Client	โปรแกรมฝั่งผู้ใช้ มีทั้งแบบหน้าจอก (gui_client.py) และแบบบรรทัดคำสั่ง (client.py) ใช้เลือกเหรียญที่ติดตาม ตั้งการแจ้งเตือน และพัก/ต่อการรับข้อมูล
ผู้ใช้เป้าหมาย	ผู้ที่ต้องเฝ้าราคาหลายเหรียญพร้อมกันและต้องการรู้ทันทีเมื่อราคาถึงระดับที่กำหนด โดยไม่ต้องนั่งจ้องหน้าจอเอง



รูปที่ 1 แอปฝั่งผู้ใช้ขณะติดตาม BTCUSDT ทุก 1 วินาที และ ETHUSDT ทุก 5 วินาทีพร้อมกัน พร้อมการแจ้งเตือนที่ทำงานแล้ว (แถวสถานะ TRIGGERED และแบนเนอร์ด้านล่างขวา) ช่องล่างสุดคือ log ของ message CMSP ทุกอันที่รับและส่ง

สิ่งที่ผู้ใช้ทำได้: เพิ่ม/เอาเหรียญออก พร้อมเลือกความถี่ที่ต้องการรับ (1, 5 หรือ 10 วินาที) **แยกอิสระต่อเหรียญ**, ตั้งและลบการแจ้งเตือนเมื่อราคาทะลุระดับที่กำหนด, พักการแสดงราคาชั่วคราวโดยไม่เสียการติดตาม (เมื่อกลับมาระบบบอกด้วยว่าพลาดไปกี่ครั้ง) และดูสถิติของ server

1.3 ขอบเขต

อยู่ในขอบเขต	ไม่อยู่ในขอบเขต
โปรโตคอลระหว่าง client กับ server ทั้งหมด — ออกแบบเอง implement เอง ทดสอบเอง	การเข้ารหัส TLS (ระบุเป็นข้อจำกัดที่รู้ตัวในหัวข้อ 6.3)
client ที่ใช้เฝ้าราคาได้จริง ทั้งแบบหน้าจอกและบรรทัดคำสั่ง	หน้าเว็บและแอปมือถือ
โหม่งจำลองข้อมูลที่ทำงานได้ครบทุกฟีเจอร์โดยไม่ต้องต่ออินเทอร์เน็ต	ฐานข้อมูล — เก็บสถานะในหน่วยความจำทั้งหมด

Binance ทำหน้าที่เป็น **แหล่งตัวเลข** เท่านั้น การเชื่อมต่อไปหา Binance ถูกจำกัดให้อยู่ในไฟล์เดียว (**upstream.py**) และอยู่หลัง server เสมอ — ช่องทางระหว่าง CMSP client กับ CMSP server ซึ่งเป็นสิ่งที่โครงการนี้ออกแบบ ยังเป็น raw socket ทั้งหมด

2. คุณลักษณะของแอปพลิเคชัน (Application characteristics)

ก่อนเลือกที่จะใช้บริการแบบใดของชั้น transport ต้องระบุก่อนว่าแอปพลิเคชันนี้ต้องการอะไรจากชั้นล่าง

ด้าน	ความต้องการของแอปพลิเคชันนี้	เหตุผลและผลต่อการออกแบบ
Data loss ทนการสูญหายได้แค่ไหน	ทนไม่ได้เลย ต้องได้ครบและเรียงลำดับถูกต้อง	ราคาที่ยา่งไปทำให้ผู้ใช้ตัดสินใจผิด และคำสั่งควบคุม (เช่นการสมัครรับข้อมูลหรือการตั้งเตือน) ที่หายไปทำให้สถานะของสองฝั่งไม่ตรงกันถาวร นอกจากนี้ข้อความที่ขาดครึ่งจะทำให้ตัวแยกข้อความพังทั้งสาย
Timing ต้องเร็วแค่ไหน	ต้องการ ความสดของข้อมูลระดับต่ำกว่า 1 วินาที แต่ไม่ใช่ระบบเวลาจริงแบบมีเส้นตายเด็ดขาด	ผู้ใช้ต้องรู้ทันตอนราคาขยับแรง แต่ช้าไป 100 มิลลิวินาทีก็ไม่ถึงกับเสียหาย จึงเลือกให้ server ส่งทันทีที่มีข้อมูลใหม่ แทนการให้ผู้ใช้ถามเอง (วัดผลจริงในหัวข้อ 6.1)
Throughput ปริมาณข้อมูล	ต่ำมาก ราว 100–200 bytes ต่อการอัปเดตหนึ่งครั้งต่อผู้ใช้หนึ่งคน สูงสุด 1 ครั้งต่อวินาทีต่อเหรียญ	คอขวดไม่ใช่ bandwidth แต่เป็น จำนวน connection ที่ต้องเปิดค้างไว้ จึงออกแบบให้ควบคุมอัตราการส่งรายผู้ใช้ และรวมการเชื่อมต่อขาออกด้วยการนับผู้ใช้ต่อเหรียญ
Security ความปลอดภัย	ต้องยืนยันตัวตนก่อนใช้งาน แต่ตัวข้อมูลราคาเป็นข้อมูลสาธารณะ	มีขั้นตอนยืนยันตัวตนบังคับก่อนใช้คำสั่งอื่น แต่ยังไม่ใช้ TLS ซึ่งเป็นข้อจำกัดที่ระบุไว้ตรงๆ ในหัวข้อ 6.3
จำนวนผู้ใช้	หลัก สิบถึงร้อย connection พร้อมกันต่อ server เครื่อง	ที่ระดับนี้โมเดลหนึ่งเทรดต่อหนึ่งผู้ใช้ อ่านง่ายและเพียงพอ ถ้าต้องรองรับหลักหมื่นต้องเปลี่ยนเป็น event loop

เมื่อรวมทุกด้านเข้าด้วยกัน จะได้ข้อสรุปว่าแอปพลิเคชันนี้ต้องการบางอย่างจากชั้นล่าง:

- **ช่องทางที่เชื่อถือได้** เพราะทั้งคำสั่งควบคุมและข้อมูลราคาห้ามหายและห้ามสลับลำดับ
- **connection ที่คงอยู่** เพราะ server ต้องส่งข้อมูลหาผู้ใช้ได้เองตลอดเวลา ไม่ใช่ตอบเฉพาะเมื่อถูกถาม
- **server ที่มีสถานะ** เพราะต้องจำว่าใครสนใจอะไร และต้องเฝ้าเงื่อนไขการแจ้งเตือนให้

สามข้อนี้คือที่มาโดยตรงของการเลือก transport layer ในหัวข้อถัดไป

3. การเลือก service model ของ Transport Layer

3.1 TCP หรือ UDP

บริการที่ต้องการ	TCP ให้อะไร	UDP ให้อะไร	ผลต่อโปรแกรมนี้
ส่งถึงแน่นอน (reliable delivery)	มี — ตรวจสอบการสูญหายแล้วส่งซ้ำให้เอง	ไม่มี	ถ้าใช้ UDP ต้องเขียน ACK, การส่งซ้ำ และ timeout เองทั้งหมด ซึ่งเท่ากับเขียน TCP ขึ้นมาใหม่แบบด้อยกว่า
รักษาลำดับ (ordering)	มี	ไม่มี	ราคาที่มาสลับลำดับทำให้ผู้ใช้เห็นราคาเก่าทับราคาใหม่ และทำให้เลขลำดับ Seq ที่ใช้ตรวจข้อมูลขาดช่วงไร้ความหมาย
เชื่อมต่อค้างไว้ได้ (connection-oriented)	มี — มี handshake และมีสถานะของ connection	ไม่มีแนวคิดนี้	ต้องผู้รายการติดตามและการแจ้งเตือนไว้กับ connection หนึ่งๆ ถ้าไม่มี connection ต้องประดิษฐ์ session id ขึ้นมาเอง
Flow control / congestion control	มี	ไม่มี	เมื่อผู้ใช้รับช้า TCP จะชะลอให้เอง (เห็นได้จากค่า window ในรูปที่ 7) ถ้าใช้ UDP ข้อมูลจะถูกทิ้งเฉยๆ
ขอบเขตข้อความ (message boundary)	ไม่มี — เป็น byte stream	มี (1 datagram = 1 ข้อความ)	นี่คือสิ่งเดียวที่ UDP ได้เปรียบ และเป็นเหตุผลที่ต้องกำหนดขอบเขตข้อความเองในหัวข้อ 4.3

สรุปการตัดสินใจ: เลือก TCP

เพราะแอปพลิเคชันนี้ต้องการความถูกต้องและลำดับของทั้งคำสั่งควบคุมและข้อมูลราคา และต้องการ connection ที่คงอยู่เพื่อให้ server ส่งข้อมูลหา client ได้เอง ซึ่ง UDP ไม่มีให้ทั้งสามอย่าง การเขียนกลไกเหล่านี้เองบน UDP จะได้ผลด้อยกว่าและใช้เวลามากกว่า

ราคาที่ต้องจ่ายคือ TCP ไม่รักษารอบขอบเขตข้อความ จึงต้องออกแบบวิธีตัดข้อความเองที่ชั้น application ข้อนี้ไม่ใช่ข้อเสียที่ซ่อนไว้ แต่ถูกพิสูจน์ด้วย packet capture ในหัวข้อ 5 ว่าเกิดขึ้นจริงบนสายและถูกจัดการถูกต้อง

3.2 ทำไมไม่ใช่ HTTP ที่มีอยู่แล้ว

HTTP/1.1 เป็น request/response แบบ stateless — server ตอบได้ก็ต่อเมื่อถูกถาม และไม่จำว่าใครสนใจอะไร สองข้อนี้ขัดกับคุณลักษณะหลักในหัวข้อ 2 โดยตรง ถ้าจะให้ HTTP ทำงานแบบส่งเองได้ต้องพึ่งกลไกเสริมอย่าง long polling, Server-Sent Events หรืออัปเกรดเป็น WebSocket ซึ่งทั้งหมดคือการยอมรับว่า HTTP เพียงๆ ทำไม่ได้ โครงการนี้จึงออกแบบโปรโตคอลใหม่ที่มีความสามารถนี้เป็นพื้นฐานตั้งแต่ต้น แล้ววัดผลเทียบกันด้วยตัวเลขจริงในหัวข้อ 6

เหตุผลที่ต้องมี server กลางแทนที่จะให้ client แต่ละคนต่อ Binance เอง มีสามข้อ: (1) ผู้ใช้หลายคนที่คุณเหรียญเดียวกันใช้การเชื่อมต่อขาออกเพียงเส้นเดียว (วัดจริงในหัวข้อ 6.1) (2) งานเฝ้าเงื่อนไครราคาถูกย้ายมาไว้ที่ server จึงทำงานแม้ผู้ใช้จะสั่งพักการแสดงผลอยู่ (3) ผู้ใช้แต่ละคนเลือกอัตราการรับข้อมูลของตัวเองได้โดยไม่กระทบคนอื่น

4. CMSP/1.0 — Application-Layer Protocol ที่ออกแบบ

4.1 ชื่อและภาพรวม

ชื่อโปรโตคอล	CMSP — Crypto Market Streaming Protocol
เวอร์ชัน	CMSP/1.0 ปรากฏใน start line ของทุก message
Transport	TCP พอร์ตมาตรฐานของโครงการ 9009
รูปแบบ	ข้อความล้วน (UTF-8) แบบ start line + headers + บรรทัดว่าง
ทิศทาง	request/response และ message ที่ server ส่งเองโดยไม่มี request นำ

ปรัชญาการออกแบบ — ยืมโครงสร้างมาจาก HTTP/1.1 โดยตั้งใจ ด้วยเหตุผลสามข้อ: (1) **อ่านออกด้วยตา** เปิด Wireshark แล้วเห็นบทสนทนาทั้งหมดเป็นข้อความ ตรวจสอบง่าย (พิสูจน์ในรูปที่ 8) (2) **ขยายได้โดยไม่พังของเดิม** การเพิ่ม header ใหม่ไม่ทำให้ฝั่งที่ไม่รู้จักมันพัง เพราะตัวแยกข้อความอ่านที่ละบรรทัดและข้ามสิ่งที่ไม่รู้จัก (3) **ตาราง status code ที่คุ้นเคย** 2xx/4xx/5xx สื่อความหมายทันที ส่วนสิ่งที่ **เพิ่มเข้ามาใหม่** และไม่มีใน HTTP คือ **status class 1xx สำหรับข้อความที่ server ส่งเอง**

4.2 รูปแบบ message

SUB CMSP/1.0	CRLF	start line: คำสั่ง + เวอร์ชัน
Symbol: BTCUSDT	CRLF	header (Key: Value)
Interval: 1s	CRLF	มีได้หลายบรรทัด
	CRLF	บรรทัดว่าง = จุดจบของ message delimiter จริงคือ <code>\r\n\r\n</code>

ทั้ง message นี้เมื่อส่งจริงคือ byte ชุดเดียว:

```
b'SUB CMSP/1.0\r\nSymbol: BTCUSDT\r\nInterval: 1s\r\n\r\n'
```

รูปที่ 2 โครงสร้าง message ของ CMSP — ไม่มี body ข้อมูลทั้งหมดอยู่ใน header และปิดท้ายด้วยบรรทัดว่างเสมอ

Encoding	UTF-8
ตัวจบบรรทัด	<code>\r\n</code> ตอนส่งเสมอ ตอนรับยอมรับทั้ง <code>\r\n</code> และ <code>\n</code> เพื่อให้ทดสอบด้วย nc /telnet ได้
ตัวค้นข้อความ	<code>\r\n\r\n</code> (บรรทัดว่าง)
ขนาดสูงสุดต่อ 1 message	8,192 bytes นับรวมตัวค้น เกินกว่านี้ตอบ 413 แล้วปิด connection
ชื่อ header	อ่านแบบไม่สนใจตัวพิมพ์ใหญ่เล็ก และห้ามค่ามี CR/LF (กันการปลอม header)

ทำไมไม่ใช่ JSON หรือ binary — JSON ต้องมีตัวแยกสองชั้น (แยกข้อความก่อน แล้วค่อย parse JSON) และอ่านใน Wireshark ยากกว่า ส่วน binary ประหยัด byte กว่าจริงแต่ debug ยากและอธิบายไม่ได้ งานนี้เลือกข้อความล้วนเพราะความชัดเจนและตรวจสอบได้สำคัญกว่า และแม้เป็นข้อความล้วนก็ยังใช้ byte น้อยกว่า HTTP polling เกือบสองเท่าตามผลวัดในหัวข้อ 6.1

4.3 การกำหนดขอบเขตข้อความ (message framing)

TCP ส่งข้อมูลเป็น byte stream ไม่ใช่ชุดข้อความ สิ่งที่คุณส่งเขียนด้วย `sendall()` หนึ่งครั้ง ไม่ได้แปลว่าผู้รับจะได้ครบพอดีหนึ่งครั้งด้วย `recv()` — เกิดได้ทั้งสองทิศทางตามรูปที่ 3

กรณี ก — 1 recv() ได้หลาย message (เกิดขึ้นจริง รูปที่ 6)

PING...\r\n\r\n	STATS...\r\n\r\n	LIST...\r\n\r\n
-----------------	------------------	-----------------

recv() ครั้งเดียวได้ทั้งก้อน 63 bytes — โค้ดที่ parse ตรงๆ จะเห็นแค่ PING แล้วทิ้งอีกสองอัน

กรณี ข — 1 message ถูกแบ่งหลาย recv()

SUB CMSP/1.0\r\n	Symbol: BTC	USDT\r\n\r\n
recv() #1	recv() #2	recv() #3

ต้องรอจนครบก่อนถึงจะ parse ได้

วิธีแก้: buffer loop ตัวเดียวจัดการได้ทั้งสองกรณี

```
buffer += chunk
while b"\r\n\r\n" in buffer:
    frame, buffer = buffer.split(b"\r\n\r\n", 1)
    → ประมวลผล frame
```

รูปที่ 3 ปัญหาขอบเขตข้อความทั้งสองทิศทางและวิธีแก้ด้วย buffer loop เดียว

โค้ดจริงที่ทำหน้าที่นี้อยู่ใน `protocol.py` และถูกใช้ร่วมกันทั้งฝั่ง server และ client จึงไม่มีทางที่สองฝั่งจะตัดข้อความไม่ตรงกัน:

```
# protocol.py - FrameBuffer.feed_raw()
if chunk:
    self.buffer += chunk

frames = []
while DELIMITER in self.buffer:
    # วนจนไม่เหลือ message ที่สมบูรณ์
    raw, self.buffer = self.buffer.split(DELIMITER, 1)
    if len(raw) + len(DELIMITER) > self.max_size:
        self.buffer = b""
        raise MessageTooLarge(...)
    frames.append(raw)
    # -> ดอน 413 แล้วปิด connection

# ยังไม่เจอจุดจบ แต่ buffer โตเกินเพดานแล้ว = ผู้ส่งไม่ยอมปิดข้อความ
if len(self.buffer) + len(DELIMITER) > self.max_size:
    self.buffer = b""
    raise MessageTooLarge("no delimiter within %d bytes" % self.max_size)

return frames
```

ข้อผิดพลาดที่พบบ่อยและถูกป้องกันไว้แล้ว

เขียนเป็น msg = recv(4096) แล้วแยกข้อความตรงๆ จะทำงาน "ถูก" ตอนทดสอบที่ละคำสั่ง แต่จะพังทันทีที่ผู้ใช้ยิงหลายคำสั่งติดกัน — จากหลักฐานในรูปที่ 6 คำสั่งสามตัวเดินทางมาใน TCP segment เดียวขนาด 63 bytes ถ้าไม่มี buffer loop จะได้คำสั่งแรกถูกอันเดียว แล้วอีกสองคำสั่งหายเงียบโดยไม่มี error ให้เห็น

4.4 Request messages (client → server)

start line ของ request คือ <COMMAND> CMSP/1.0 มีทั้งหมด 11 คำสั่ง

คำสั่ง	header บังคับ	ไม่บังคับ	ความหมาย → response ที่เป็นไปได้
HELLO	Client-Name	—	เปิดบทสนทนา บอกชื่อ client → 200 / 400 / 426
AUTH	User , Token	—	ยืนยันตัวตน → 200 / 401
SUB	Symbol	Interval	ขอรับราคาของเหรียญนั้น → 201 / 404 / 409 / 429 / 503
UNSUB	Symbol	—	เลิกรับ → 202 / 410
ALERT	Action=SET + Symbol , Condition , Value Action=DEL + Alert-Id	—	ตั้ง/ลบการแจ้งเตือน → 203 / 204 / 404 / 411 / 429
LIST	Type (SUB หรือ ALERT)	—	ดูรายการของตัวเอง → 200 / 400
STATS	—	—	สถิติของ server → 200
PAUSE	—	—	หยุดรับราคาชั่วคราวโดยไม่เสียรายการติดตาม → 207 / 410
RESUME	—	—	กลับมารับราคา พร้อมบอกจำนวนครั้งที่พลาด → 208 / 400
PING	—	—	ทดสอบว่าสายยังดี → 205
QUIT	—	—	ปิดบทสนทนา → 206 แล้ว server ปิด connection

ตัวอย่าง request จริงที่จับได้จากสาย (frame 8 และ 10 ของไฟล์ capture): AUTH CMSP/1.0\r\nUser: student\r\nToken: 1234\r\n\r\n และ SUB CMSP/1.0\r\nSymbol: BTCUSDT\r\nInterval: 1s\r\n\r\n

4.5 Response messages (server → client)

start line ของ response คือ CMSP/1.0 <code> <PHRASE> — มีทั้งเลขและคำอธิบายเสมอ เพื่อให้อ่าน log ได้โดยไม่ต้องเปิดตาราง

Code	header ที่แนบมาด้วย
200 OK	ขึ้นกับคำสั่ง — Client-Name (HELLO), User (AUTH), Type + Count + Item-N (LIST), Uptime + Clients + Upstream-Symbols + Messages-Sent (STATS)
201 SUBSCRIBED	Symbol , Interval , Active-Subs
202 UNSUBSCRIBED	Symbol , Active-Subs
203 ALERT CREATED	Alert-Id , Symbol , Condition , Value
204 ALERT DELETED	Alert-Id
205 PONG	Timestamp
206 GOODBYE	Detail
207 STREAM PAUSED	Detail
208 STREAM RESUMED	Missed-Ticks — จำนวนครั้งที่ควรได้รับแต่ถูกกลืนไว้ระหว่างพัก
4xx / 5xx	Detail เสมอ บอก header บริบท เช่น Symbol , Interval , Value

การส่งรายการหลายชิ้นใน message เดียว — เนื่องจากไม่มี body การตอบ LIST จึงใช้รูปแบบ Count + Item-N เช่น Count: 2 ตามด้วย Item-1: id=1 BTCUSDT ABOVE 63020.43 status=TRIGGERED และ Item-2: ... ข้อดีคือใช้ตัวแยกข้อความตัวเดิมได้ทันทีและอ่านออกใน Wireshark ข้อเสียที่ยอมรับคือเปลืองพื้นที่ header และแพดาน 8,192 bytes จำกัดจำนวนรายการ ซึ่งไม่เป็นปัญหาเพราะกำหนดแพดานไว้ที่ 5 รายการติดตามและ 10 การแจ้งเตือนต่อผู้ใช้อยู่แล้ว

4.6 Push messages — สิ่งที่ HTTP ทำไม่ได้

นี่คือความสามารถที่ทำให้ CMSP ต่างจาก HTTP โดยพื้นฐาน: server ส่งข้อความหา client ได้เองเมื่อไรก็ได้ตราบใดที่ connection ยังเปิดอยู่ ในโค้ดแยกเป็นคนละฟังก์ชันกับการตอบ request และแยก log กันเป็น PUSH → กับ SEND → เพื่อให้ตรวจสอบได้ว่าข้อความไหนเกิดจากอะไร

Code	เกิดจาก	header
110 DATA	มีราคาใหม่จาก upstream และถึงรอบ Interval ที่ผู้ใช้ขอไว้	Symbol , Price , Change24h , Seq , Timestamp
111 ALERT TRIGGERED	ราคาชนเงื่อนไขที่ผู้ใช้ตั้งไว้ (ส่งเฉพาะเจ้าของและส่งแม้กำลังส่งพักอยู่)	Alert-Id , Symbol , Condition , Threshold , Price , Timestamp
112 SERVER NOTICE	แหล่งข้อมูลต้นทางหลุดหรือกลับมา	Detail
503 / 504	แหล่งข้อมูลใช้ไม่ได้ / เียบเกินเวลาที่กำหนด	Detail

ตัวอย่างที่จับได้จริงจากสาย: CMSP/1.0 110 DATA\r\nSymbol: BTCUSDT\r\nPrice: 63040.58\r\nChange24h: 0.06\r\nSeq: 1\r\nTimestamp: 1788096572877\r\n\r\n

4.7 Status codes

มีทั้งหมด 25 ตัว แบ่งเป็น 4 class โดยยืมความหมายของ 2xx/4xx/5xx มาจาก HTTP แล้วเพิ่ม 1xx ขึ้นมาใหม่

Class	ความหมาย	Code ที่มี
1xx	server ส่งเองโดยไม่มี request นำ	110 DATA, 111 ALERT TRIGGERED, 112 SERVER NOTICE
2xx	คำสั่งสำเร็จ	200 OK ถึง 208 STREAM RESUMED รวม 9 ตัว
4xx	ความผิดพลาด client แก้แล้วลองใหม่ได้	400 , 401 , 404 , 405 , 409 , 410 , 411 , 413 , 426 , 429
5xx	ความผิดพลาด server หรือแหล่งข้อมูล client แก้เองไม่ได้	500 , 503 , 504

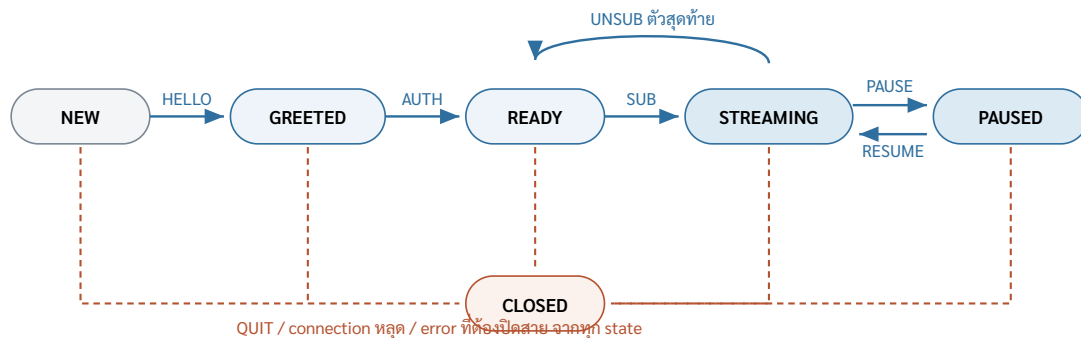
การแบ่งตาม ใครต้องเป็นคนแก้ ทำให้ client เขียนการจัดการข้อผิดพลาดได้โดยไม่ต้องรู้จัก code ทุกตัว เจอ 4xx แปลว่าคำสั่งที่ส่งไปผิดพลาดต้องแก้ก่อนส่งใหม่ เจอ 5xx แปลว่ารอแล้วลองใหม่ได้ ส่วน 1xx ต้องไม่ถูกนำไปจับคู่กับ request ที่ค้างอยู่ เพราะไม่ใช่คำตอบของอะไรเลย ตารางฉบับเต็มพร้อมเงื่อนไขการเกิดของทุก code อยู่ในภาคผนวก

4.8 นิยาม field และค่าที่ยอมรับ

Field	ค่าที่ยอมรับ	ผิดแล้วได้อะไร / หมายเหตุ
Symbol	ตรงกับ <code>^[A-Z0-9]{5,12}\$</code> และต้องมีอยู่จริงบนแหล่งข้อมูล	รูปแบบผิด → 400 , รูปแบบถูกแต่ไม่มีจริง → 404
Interval	1s , 5s , 10s — ไม่ระบุถือเป็น 1s	ค่าอื่น → 400 พร้อมบอกค่าที่รองรับใน Detail
Condition	ABOVE หรือ BELOW	ค่าอื่น → 400
Value	ทศนิยมบวก	ค่าติดลบหรือแปลงเป็นตัวเลขไม่ได้ → 400
Timestamp	Unix epoch หน่วย มิลลิวินาที	ใช้คำนวณความสดของข้อมูลในหัวข้อ 6.1
Seq	จำนวนเต็มต่อเหรียญ เริ่มที่ 1 เพิ่มทีละ 1 ทุกครั้งที่ upstream ส่งข้อมูลมา	ดูคำอธิบายด้านล่าง
เพดานต่อผู้ใช้	ติดตาม 5 เหรียญ และตั้งเดือน 10 รายการ	เกิน → 429 พร้อมบอกเพดานใน Detail

Seq เข้าซ้อนกับ sequence number ของ TCP หรือไม่ — ไม่เข้า เพราะนับคนละอย่างและอยู่คนละชั้น: sequence number ของ TCP นับ byte ที่ส่งไปในสายเพื่อตอบว่าข้อมูลถึงครบและเรียงถูกหรือไม่ ซึ่ง TCP จัดการเองจนแอบมองไม่เห็น ส่วน Seq ของ CMSP นับ *ครั้งที่ราคาเปลี่ยน* ต่อเหรียญ เพื่อบอกผู้ใช้ว่ามีข้อมูลกี่ครั้งที่ server ตั้งใจไม่ส่ง ตามอัตราที่ผู้ใช้เลือกไว้ ตัวอย่างจากการรันจริง: ผู้ใช้ที่ขอ Interval: 1s ได้รับ Seq เป็น 1, 5, 9, 13 ... เพราะแหล่งข้อมูลส่งมา 4 ครั้งต่อวินาที ในมุมมองของ TCP ไม่มี byte ไหนหายไปเลย จึงบอกเรื่องนี้ไม่ได้

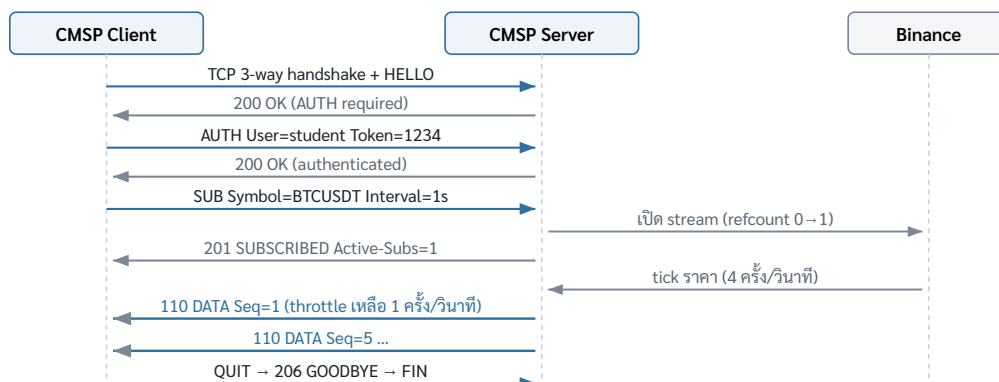
4.9 State machine



รูปที่ 4 สถานะของ connection หนึ่งเส้น — เส้นที่บ่งชี้การเปลี่ยนสถานะปกติ เส้นประคือการปิดสายซึ่งเกิดได้จากทุกสถานะ

สถานะ	คำสั่งที่ใช้ได้	ส่งอย่างอื่นแล้วได้อะไร
NEW	HELLO , QUIT	400 BAD REQUEST (say HELLO first)
GREETED	AUTH , PING , QUIT	คำสั่งที่รู้จักแต่ยังไม่ยืนยันตัวตน → 401 ; คำสั่งที่ไม่รู้จัก → 405
READY	ทุกคำสั่ง ยกเว้น PAUSE / RESUME ที่ต้องมีรายการติดตามก่อน	PAUSE ตอนไม่มีรายการ → 410 ; RESUME ตอนไม่ได้พัก → 400
STREAMING	ทุกคำสั่ง	—
PAUSED	ทุกคำสั่ง — ยังได้รับ 111 และ 112 แต่ไม่ได้รับ 110	—

4.10 ตัวอย่างการสื่อสารตั้งแต่ต้นจนจบ



รูปที่ 5 ลำดับการสื่อสารเมื่อทุกอย่างปกติ — สังเกตว่า 201 ถูกส่งก่อน 110 เสมอ และการเชื่อมต่อไปยังแหล่งข้อมูลถูกเปิดเมื่อมีผู้สนใจรายแรกเท่านั้น เมื่อผู้ใช้คนสุดท้ายเลิกติดตามจึงปิด

5. การตรวจสอบด้วย Wireshark

log ที่โปรแกรมพิมพ์ออกมาเป็นสิ่งที่ผู้พัฒนาเขียนเอง ถ้าเขียนผิดก็พิมพ์ผิดตาม มันพิสูจน์ได้แค่ว่าได้เดินมาถึงบรรทัดนั้น หัวข้อนี้จึงใช้ Wireshark ซึ่งดักฟังจากภายนอกโปรแกรม มายืนยันว่า byte ที่ออกไปบนสายเป็นไปตามที่ออกแบบไว้จริง

วิธีเก็บหลักฐาน: Wireshark 4.x กับ Npcap (เปิด loopback support) จับที่ \Device\NPF_Loopback ด้วย capture filter tcp port 9009 ทั้งหมดทำซ้ำได้ด้วยสคริปต์ที่อยู่ในโฟลเดอร์โครงงาน

5.1 หลักฐานที่ 1 – TCP ไม่รักษาขอบเขตข้อความจริง

No.	Time	Source	Destination	Protocol	Length	Source Port	Info
12	0.988115	127.0.0.1	127.0.0.1	TCP	187	51168	51168 → 9009 [PSH, ACK] Seq=84 Ack=120 Win=65280 Len=63
20	1.954599	127.0.0.1	127.0.0.1	TCP	187	51168	51168 → 9009 [PSH, ACK] Seq=147 Ack=289 Win=65824 Len=63

Section number: 1	0000 02 00 00 00 45 00 00 67 b8 ad 40 00 00 00 43 c1E..@...C- 0010 7f 00 00 01 7f 00 00 01 c7 00 23 31 99 fd 80 66f.....f 0020 66 86 b1 d2 50 18 00 ff 8f f1 00 00 50 49 4e 47 f...P.....PING 0030 20 43 44 53 50 2f 31 2e 30 0d 0a 0d 53 54 41 CMSP/1.0...STA 0040 54 53 20 43 4d 53 50 2f 31 2e 30 0d 0a 0d 4c TS CMSP/1.0...L 0050 49 53 54 20 43 4d 53 50 2f 31 2e 30 0d 0a 54 79 TST CMSP/1.0...Ty 0060 70 65 3a 20 53 55 42 0d 0a 0a 0a 0a 0a 0a 0a per: SUB.....
Frame 12: Packet, 187 bytes on wire (856 bits), 187 bytes captured (856 bits) on interface \Device\NPF_Loopback, Id 0	
Section number: 1	
> Interface id: 0 (\Device\NPF_Loopback)	
Encapsulation type: NULL/Loopback (15)	
Arrival Time: Aug 30, 2026 22:12:03.248526300 SE Asia Standard Time	
UTC Arrival Time: Aug 30, 2026 15:12:03.248526300 UTC	
Epoch Arrival Time: 1788102723.248526300	
[Time shift for this packet: 0.000000000 seconds]	
[Time delta from previous captured frame: 115.300 microseconds]	
[Time since reference or first frame: 2.129700 milliseconds]	
Frame Number: 12	
Frame Length: 187 bytes (856 bits)	
Capture Length: 187 bytes (856 bits)	
[Frame is marked: False]	
[Frame is ignored: False]	
[Protocols in frame: null:ip:tcp]	
Character encoding: ASCII (0)	
[Coloring Rule Name: TCP]	
[Coloring Rule String: tcp]	
> Null/Loopback	
> Internet Protocol Version 4, Src: 127.0.0.1, Dst: 127.0.0.1	
> Transmission Control Protocol, Src Port: 51168, Dst Port: 9009, Seq: 84, Ack: 120, Len: 63	
Source Port: 51168	
Destination Port: 9009	
[Stream Index: 0]	
[Stream Packet Number: 12]	
> [Conversation completeness: Complete, WITH_DATA (31)]	
> [TCP Segment Len: 63]	
Sequence Number: 84 (relative sequence number)	
Sequence Number (raw): 2583531110	
[Next Sequence Number: 147 (relative sequence number)]	
Acknowledgment Number: 120 (relative ack number)	
Acknowledgment number (raw): 1720183378	
0101 = Header Length: 20 bytes (5)	
> Flags: 0x018 (PSH, ACK)	
Window: 255	
[Calculated window size: 65280]	
[Window size scaling factor: 256]	
Checksum: 0xb8ff (unverified)	
[Checksum Status: Unverified]	
Urgent Pointer: 0	
> [Timestamps]	
> [SEQ/ACK analysis]	
[Client Contiguous Streams: 1]	
[Server Contiguous Streams: 1]	
TCP payload (63 bytes)	
TCP segment data (63 bytes)	

รูปที่ 6 TCP segment เดียวขนาด [TCP Segment Len: 63] บรรจุ message ของ CMSP ครบ 3 อัน คือ PING, STATS และ LIST (เห็นได้ในช่อง Packet Bytes ด้านขวา) สิ่งที่พิสูจน์: ผู้รับต้องกำหนดขอบเขตข้อความเองที่ชั้น application ตามที่ออกแบบไว้ในหัวข้อ 4.3

ภาพนี้เกิดจากการสั่งให้ client ยิงคำสั่งสามตัวใน sendall() ครั้งเดียว ผลที่ได้คือ payload ขนาด 63 bytes หน้าตาแบบนี้:

```
b'PING CMSP/1.0\r\n\r\nSTATS CMSP/1.0\r\n\r\nLIST CMSP/1.0\r\n\r\nType: SUB\r\n\r\n\r\n'
```

หลักฐานฝั่งผลลัพธ์คือ server ตอบกลับมาเป็น 3 response แยกกัน (frame 14, 16, 18 ได้แก่ 205 PONG, 200 OK ของ STATS และ 200 OK ของ LIST) ซึ่งเป็นไปไม่ได้ถ้าแยกข้อความไม่ออก

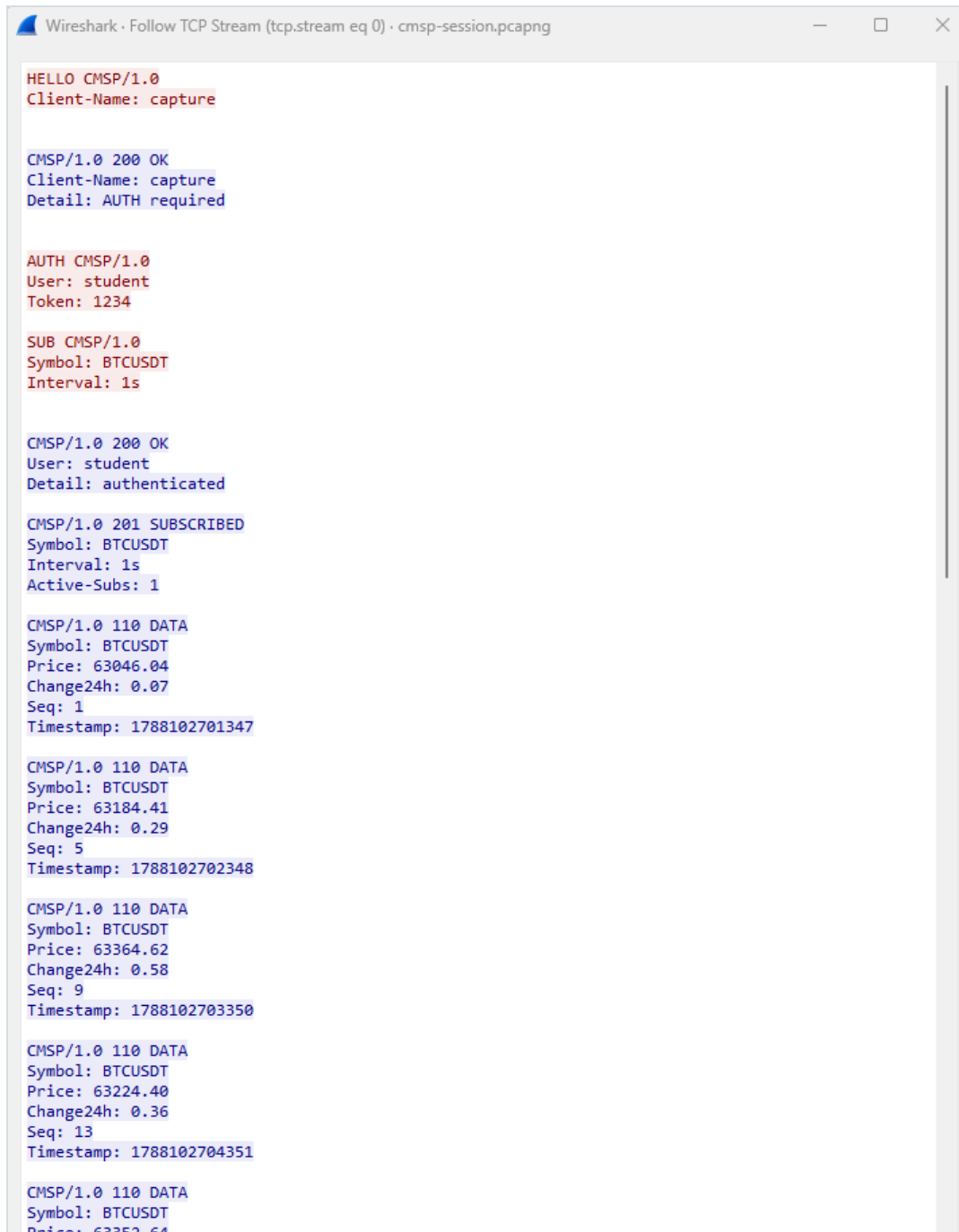
กรณีกลับกัน (1 message ถูกแบ่งหลาย segment) บังคับให้เกิดบน loopback ไม่ได้ เพราะ MSS ของ loopback คือ 65,495 bytes ซึ่งใหญ่กว่าเพดาน message ของ CMSP (8,192 bytes) ถึง 8 เท่า รายงานนี้จึงอธิบายด้วยแผนภาพ (รูปที่ 3 กรณี ข) และระบุตรงๆ ว่าจับภาพจริงไม่ได้เพราะเหตุใด อย่างไรก็ตามได้ตรงรองรับกรณีนี้แล้ว และถูกทดสอบด้วย unit test ที่ป้อน message เดียวแบ่งเป็นสาม chunk และป้อนทีละ byte

5.2 หลักฐานที่ 2 – เป็น connection-oriented จริงตามที่เราเลือกไว้

1	127.0.0.1	127.0.0.1	TCP	56	64264	64264 → 9009 [SYN] Seq=0 Win=65535 Len=0 MSS=65495 WS=256 SACK_PERM
2	0.000108	127.0.0.1	127.0.0.1	TCP	56	9009 → 64264 [SYN, ACK] Seq=0 Ack=1 Win=65535 Len=0 MSS=65495 WS=256 SACK_PERM
3	0.000029	127.0.0.1	127.0.0.1	TCP	44	64264 → 9009 [ACK] Seq=1 Ack=1 Win=65280 Len=0

รูปที่ 7 frame 1–3: [SYN] → [SYN, ACK] → [ACK] เกิดขึ้นก่อนที่ byte แรกของ HELLO จะถูกส่ง สิ่งที่พิสูจน์: CMSP ทำงานบนบริการแบบ connection-oriented ตามเหตุผลในหัวข้อ 3 ถ้าใช้ UDP จะไม่มีสามเฟรมนี้เลย และค่า Win=65535 ที่ทั้งสองฝั่งประกาศคือ receive window ที่ TCP ใช้ทำ flow control ให้โดยที่ชั้น application ไม่ต้องทำเอง

5.3 หลักฐานที่ 3 — message บนสายตรงกับสเปกในหัวข้อ 4



รูปที่ 8 Follow TCP Stream ของทั้ง session — สีแดงคือ client → server สีน้ำเงินคือ server → client **สิ่งที่พิสูจน์:** บทสนทนาบนสายจริงตรงกับรูปแบบ message ที่ออกแบบไว้ทุกประการ ทั้ง start line, header และบรรทัดว่างปิดท้าย สังเกตว่า client ส่ง AUTH กับ SUB ติดกันโดยไม่รอ response (pipelining) แล้ว server ตอบกลับตามลำดับเป็น 200 แล้ว 201 — ทำได้เพราะโปรโตคอลตัดข้อความด้วยบรรทัดว่างและประมวลผลตามลำดับที่มาถึง

ภาพนี้ยังเผยข้อจำกัดที่รู้ตัวไปพร้อมกัน: บรรทัด Token: 1234 อ่านได้ด้วยตาเปล่า เพราะ CMSP ยังไม่มี TLS ในทางกลับกัน ช่องทางที่ server ใช้คุยกับ Binance เป็น wss:// ซึ่ง Wireshark อ่านได้แค่ว่าเป็น TLSv1.3 Application Data — เป็นหลักฐานว่าปัญหานี้ไม่ใช่เรื่องทฤษฎี และแนวทางแก้คือห่อ CMSP ด้วย TLS แบบเดียวกัน ซึ่งทำได้โดยไม่ต้องแก้ตัวโปรโตคอลเลยเพราะ CMSP อยู่เหนือชั้น transport อยู่แล้ว

6. จุดเด่นของ CMSP

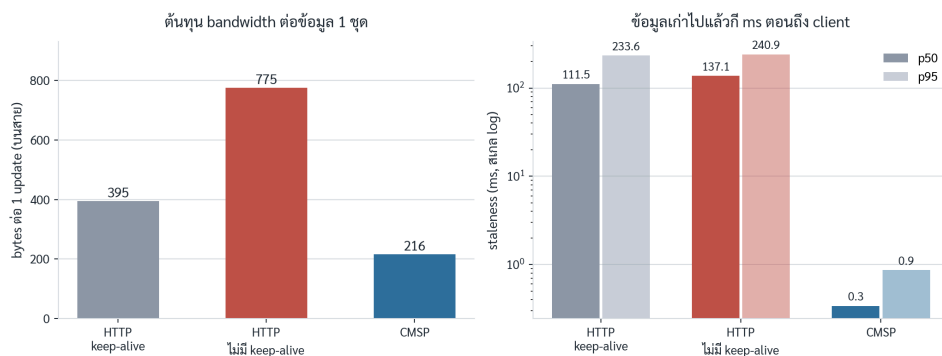
6.1 ผลการวัดเทียบกับ HTTP polling

การเทียบกับ HTTP เวอร์ชันที่จูงใจทำให้แย่งจะทำให้ตัวเลขไม่มีความหมาย baseline ของงานนี้จึงเป็น **HTTP/1.1 + keep-alive + JSON body** สั้น ซึ่งเป็นสิ่งที่ระบบจริงใช้กัน และเพิ่มขาที่สาม (ไม่มี keep-alive) เพื่อแสดงต้นทุนของการเปิด connection ใหม่ทุกครั้ง ทั้งสามขาอ่านจากแหล่งข้อมูลเดียวกัน อัตราเท่ากัน ใช้ body ที่มีข้อมูล 4 พิลด์เท่ากับที่ CMSP ส่ง และ poll ทุก 1 วินาทีเท่ากับ Interval ที่ CMSP client ขอไว้

เงื่อนไขการทดสอบ: Windows 11, Python 3.12.4, วัตบน loopback, 60 วินาทีต่อขา, 1 client, 1 เหรียญ (BTCUSDT), นับ byte ชั้น application จาก `tcp.len` และ byte รวมบนสายจาก `frame.len` ด้วย `tshark`, ความเก่าของข้อมูลคำนวณจาก `recv_time - Timestamp` ด้วยนาฬิกาเรือนเดียวกัน

ตารางที่ 1 ผลการวัดจริง 60 วินาทีต่อขา

ตัวชี้วัด	HTTP keep-alive	HTTP ไม่มี keep-alive	CMSP/1.0
Bytes ชั้น application	12,807	15,361	6,921
Bytes รวมบนสาย	23,699	47,273	13,149
จำนวน packet	247	692	141
Bytes ต่อ 1 การอัปเดต	395.0	774.9	215.6
จำนวน TCP handshake	1	61	1
ความเก่าของข้อมูล p50	111.5 ms	137.1 ms	0.3 ms
ความเก่าของข้อมูล p95	233.6 ms	240.9 ms	0.9 ms



รูปที่ 9 ข่าย: ต้นทุนต่อข้อมูล 1 ชุด — CMSP ใช้ 216 bytes เทียบกับ 395 และ 775 bytes ของ HTTP สองแบบ ขวา: ความเก่าของข้อมูลจนถึงมือผู้ใช้ (สเกล log)

ตีความตัวเลข

- เรื่อง byte** — CMSP ใช้ byte บนสายน้อยกว่า HTTP + keep-alive **1.8 เท่า** และน้อยกว่าแบบไม่มี keep-alive **3.6 เท่า** สาเหตุไม่ได้อยู่ที่การบีบข้อมูลให้เล็กลง (message ของ CMSP ก็เป็นข้อความล้วนเหมือน HTTP) แต่อยู่ที่โครงสร้างการสื่อสาร: HTTP ต้องส่งคำขอขึ้นไปทุกครั้งที่ยากได้ข้อมูล ส่วน CMSP ส่ง SUB ครั้งเดียวตอนเริ่ม จากนั้นข้อมูลไหลทางเดียวโดยไม่มีคำขอซ้ำอีกเลย
- เรื่อง handshake** — ขาที่ไม่มี keep-alive ต้องจับมือ 3 ทางใหม่ 61 ครั้งใน 60 วินาที ทำให้จำนวน packet สูงถึง 692 เทียบกับ 141 ของ CMSP ทั้งที่ได้ข้อมูลเท่ากัน และตัวเลขนี้ยังเป็นกรณีที่ไม่มี TLS ถ้าใช้ HTTPS ต้นทุนต่อการเปิด connection จะเพิ่มอีกหลาย packet ต่อครั้ง
- เรื่องความสดของข้อมูล** — จุดขายที่แท้จริง — ค่า p50 ของ HTTP อยู่ที่ 111–137 ms ไม่ใช่เพราะเครือข่ายช้า (ทดสอบบน loopback ซึ่ง RTT ต่ำกว่า 1 ms) แต่เพราะข้อมูลที่ poll ได้มานั้นรออยู่ที่ server มาแล้วโดยเฉลี่ยครึ่งหนึ่งของช่วงที่ราคาเปลี่ยน ส่วนต่างประมาณ **370 เท่า** นี้ไม่ได้เกิดจากการเขียนโค้ดเก่งกว่า แต่เกิดจากรูปแบบการสื่อสารที่ต่างกันโดยพื้นฐาน — polling ทุก T วินาทีทำให้ข้อมูลเก่าเฉลี่ย T/2 เสมอ ไม่ได้ขึ้นจากเวลา T ลงซึ่งแลกมาด้วย byte ที่เพิ่มขึ้น ส่วนการส่งเองมีความเก่าเท่ากับ RTT ของเครือข่ายเท่านั้น และไม่ขึ้นกับความถี่ของข้อมูลเลย

ผลของการรวมการเชื่อมต่อขาออก

วัดในโหมดต่อ Binance จริง โดยนับ TCP connection ของโปรเซส server ที่ปลายทางพอร์ต 9443

สถานการณ์	connection ไปหา Binance	ถ้าไม่มีการนับผู้ใช้
client 1 ตัวติดตาม BTCUSDT	1	1
client 3 ตัวติดตาม BTCUSDT	1	3
client 5 ตัวติดตาม BTCUSDT	1	5
client 5 ตัว, 2 เหรียญ	2	6
ปิด client ทั้งหมด	0	0

จำนวน connection ขาออกขึ้นกับ จำนวนเหรียญที่มีคนดู ไม่ใช่จำนวนผู้ใช้ — เป็นสิ่งที่ HTTP polling ทำแทนไม่ได้ เพราะไม่มีใครรู้ว่าใครสนใจอะไรค้างอยู่บ้าง

6.2 ความสามารถที่ต่างจาก HTTP เชิงโครงสร้าง

ความสามารถ	HTTP/1.1	CMSP/1.0
server ส่งข้อมูลเองโดยไม่ถูกถาม	ทำไม่ได้ ต้องพึ่ง SSE หรืออัปเดตเป็น WebSocket	ได้ — status class 1xx
server จำได้ว่าใครสนใจอะไร	ไม่ได้ (stateless)	ได้ — ตาราง subscription ต่อ connection
เฝ้าเงื่อนไขราคาให้ฝั่ง server	ไม่มี ผู้ใช้ต้อง poll แล้วเช็คเอง	มี — ประเมินทุกครั้งที่ราคาเปลี่ยน แล้วส่ง 111 ให้เจ้าของ
หยุด/ต่อสตรีมโดยไม่เสียรายการติดตาม	ไม่มีแนวคิดนี้	มี — PAUSE / RESUME พร้อมรายงาน Missed-Ticks
ผู้ใช้เลือกอัตราการรับข้อมูลของตัวเอง	ทำได้แต่ server ควบคุมไม่ได้	มี — Interval ต่อรายการติดตาม และ server บังคับให้
ตรวจจับว่าข้อมูลขาดช่วง	ไม่มี	มี — Seq ต่อเหรียญ
รวมผู้ใช้หลายคนให้ใช้การเชื่อมต่อขาออกเดียว	ไม่เกี่ยวข้อง	มี — ตามตารางในหัวข้อ 6.1
แจ้งสถานะของแหล่งข้อมูลต้นทาง	ตอบได้แค่ตอนถูกถาม	ได้ทันทีที่เกิดเหตุ — 112 , 503 , 504

6.3 จุดที่ HTTP ยังชนะ และข้อจำกัดที่รู้ตัว

ถ้าไม่มีหัวข้อนี้ รายงานจะกลายเป็นโฆษณา ทั้งหมดนี้เป็นข้อเสียจริงของ CMSP ที่ยังไม่ได้แก้

- **จุดคุ้มทุน** — CMSP ประหยัดกว่าเมื่อผู้ใช้ต้องการข้อมูลถี่กว่าทุกๆ ~18 วินาที แต่ถ้าต้องการนานกว่านั้น เช่น ชั่วโมงละครั้ง HTTP ชนะขาด: poll ครั้งเดียวจ่าย 775 bytes ขณะที่ CMSP ยังส่ง 360 ครั้ง/ชั่วโมงคิดเป็น ~77,600 bytes เพราะ Interval ที่ช้าที่สุดของเราคือ 10 วินาที — เป็นข้อจำกัดที่เราออกแบบเอง ไม่ใช่ข้อจำกัดของแนวคิด
- **Ecosystem** — curl, browser, Postman, load balancer, CDN, reverse proxy ใช้กับ CMSP ไม่ได้เลย ใครจะใช้ต้องเขียน client เอง
- **Scale แนวนอน** — HTTP เป็น stateless จึงกระจายโหลดข้ามเครื่องได้ทันที ส่วน CMSP เก็บรายการติดตามและการแจ้งเตือนไว้ในหน่วยความจำของ server ตัวเดียว ถ้า server สัมผัสผู้ใช้ต้องตั้งใหม่ทั้งหมด
- **ความปลอดภัย** — HTTPS เป็นมาตรฐานที่ทุกคนใช้อยู่แล้ว ส่วน CMSP ยังส่งทุกอย่างเป็น plaintext รวมถึง token (เห็นได้จากรูปที่ 8)
- **ต้นทุนการคง connection** — CMSP ต้องคง connection ไว้ตลอด ซึ่งกิน file descriptor, เธรด และหน่วยความจำต่อผู้ใช้หนึ่งคนบน server ส่วน HTTP แบบไม่มี keep-alive คืนทรัพยากรทั้งหมดทันทีที่ตอบเสร็จ

สรุปจุดเด่น

CMSP ไม่ได้ดีกว่า HTTP โดยทั่วไป แต่ดีกว่าอย่างชัดเจนสำหรับงานที่ข้อมูลเปลี่ยนแปลงบ่อยและผู้ใช้ต้องรู้ทันที ซึ่งเป็นวัตถุประสงค์ที่ตั้งไว้ตั้งแต่หัวข้อ 1 — ที่ภาระงานเดียวกัน CMSP ใช้ byte น้อยกว่า 1.8 เท่า ส่งข้อมูลถึงมือผู้ใช้เร็วกว่าประมาณ 370 เท่า รวมผู้ใช้หลายคนให้ใช้การเชื่อมต่อขาออกเส้นเดียว และมีความสามารถที่ HTTP ทำแทนไม่ได้คือการเฝ้าเงื่อนไขราคาฝั่ง server กับการพัก/ต่อสตรีมโดยไม่เสียรายการติดตาม แลกกับการต้องคง connection ไว้ตลอด การสูญเสีย ecosystem ทั้งหมดของ HTTP และการที่ยังไม่มี TLS

ภาคผนวก — ตาราง status code ทั้ง 25 ตัว

Code	Phrase	เกิดขึ้นเมื่อ
110	DATA	ถึงรอบ Interval ของผู้ใช้ และมีราคาใหม่จากแหล่งข้อมูล
111	ALERT TRIGGERED	ราคาชนเงื่อนไขของการแจ้งเตือนที่ยังทำงานอยู่ (ส่งเฉพาะเจ้าของ ส่งแม้กำลังพัก)
112	SERVER NOTICE	แหล่งข้อมูลต้นทางหลุดหรือกลับมาเชื่อมต่อได้
200	OK	สำเร็จทั่วไป — ตอบ HELLO , AUTH , LIST , STATS
201	SUBSCRIBED	SUB สำเร็จ
202	UNSUBSCRIBED	UNSUB สำเร็จ
203	ALERT CREATED	ALERT Action=SET สำเร็จ
204	ALERT DELETED	ALERT Action=DEL สำเร็จ
205	PONG	ตอบ PING
206	GOODBYE	ตอบ QUIT แล้วปิด connection
207	STREAM PAUSED	PAUSE สำเร็จ (สั่งซ้ำได้ ตอบเหมือนเดิม)
208	STREAM RESUMED	RESUME สำเร็จ พร้อม Missed-Ticks
400	BAD REQUEST	รูปแบบผิด, header ขาด, ค่าไม่ถูกต้อง, สั่งก่อน HELLO , RESUME ตอนไม่ได้พัก
401	UNAUTHORIZED	ยังไม่ยืนยันตัวตน หรือ user/token ไม่ถูกต้อง
404	SYMBOL NOT FOUND	รูปแบบเหรียญถูกต้องแต่ไม่มีอยู่บนแหล่งข้อมูล
405	UNKNOWN COMMAND	คำสั่งไม่อยู่ในรายการ 11 คำสั่ง
409	ALREADY SUBSCRIBED	SUB เหรียญที่ติดตามอยู่แล้ว
410	NOT SUBSCRIBED	UNSUB สิ่งที่ไม่ได้ติดตาม หรือ PAUSE ตอนไม่มีรายการติดตาม
411	ALERT NOT FOUND	ลบการแจ้งเตือนที่ไม่มีอยู่ หรือไม่ใช่ของผู้ใช้นั้น
413	MESSAGE TOO LARGE	message เกิน 8,192 bytes หรือไม่พบตัวค้นภายในขนาดนั้น → ปิด connection หลังตอบ
426	VERSION NOT SUPPORTED	เวอร์ชันใน start line ไม่ใช่ CMSP/1.0
429	LIMIT EXCEEDED	ติดตามเกิน 5 เหรียญ หรือตั้งเตือนเกิน 10 รายการต่อผู้ใช้
500	INTERNAL ERROR	ข้อผิดพลาดที่ไม่คาดคิดฝั่ง server — ตอบแล้ว connection ยังใช้ได้
503	UPSTREAM UNAVAILABLE	SUB ขณะแหล่งข้อมูลดับ หรือแจ้งผู้ที่ติดตามค้างไว้ตอนแหล่งข้อมูลหลุด
504	UPSTREAM TIMEOUT	ไม่มีข้อมูลใหม่จนเกินเวลาที่กำหนด (โหมดจำลอง 3 วินาที / Binance 20 วินาที)